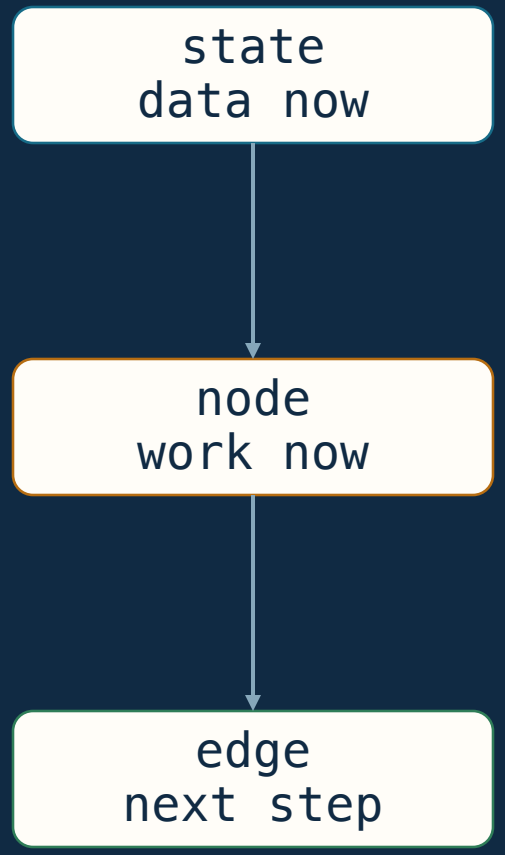
EE 599 SESSION 05

LangGraph fundamentals

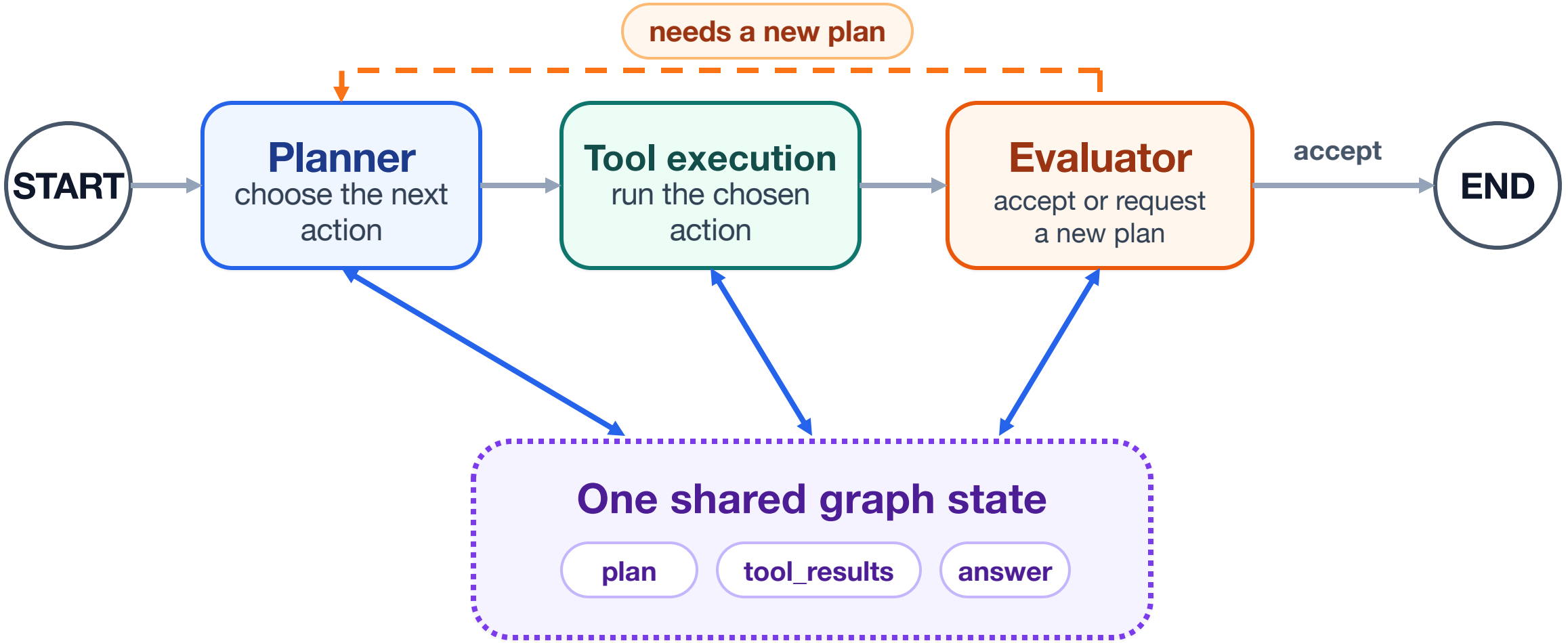
LangGraph is a runtime for building stateful, multi-step AI and agentic programs as graphs you can run, inspect, and resume.



LangGraph runs stateful workflows as inspectable graph steps

- LangGraph runs a **graph of steps**.
- Each node reads **shared state**, updates it, and uses a **routing rule** to select the next node.
- A checkpointer saves state so the workflow can **pause and resume**.

Every node reads and updates one shared graph state



Current state goes into each node; each node returns a partial update to the same state.

LangGraph runs stateful workflows as inspectable graph steps

DATA	What data do we have now?
STEP	What step runs next?
ROUTE AKA EDGE	Which rule selects the next node?
RECOVERY	What progress can we load later?

LangGraph also supports sequences, branches, parallel work, loops, and approvals.

Plain Python workflows hide state, routes, and recovery

AI WORKFLOW IN ONE FUNCTION

```
def run_job_radar(resume_text):
    profile = extract_profile(resume_text)
    jobs = search_a(profile) + search_b(profile)
    best = score(deduplicate(jobs), profile)[0]
    claim = tailor_resume(best)
    if verify(claim, profile):
        return build_preview(claim)
    return {"status": "failed"}
```

Example: résumé search-and-tailor workflow

PARALLEL WORK
search_a(...) + search_b(...) suggests independent searches, but no fork or join is declared.

STATE
profile, jobs, best, and claim live only in local variables.

ROUTING
if verify(...) chooses the next step, but the decision has no named route or trace event.

RECOVERY
Both returns end the function. Neither exposes a saved run ID or restart point.

Yes, this is a workflow, but it is encoded as one function. The runtime sees a final return value, not **named steps**, **routes**, **state**, or **checkpoints**.

Today we'll learn how LangGraph executes workflows

Learn LangGraph's state, nodes, edges, routing, and checkpoints.

1

Read

Find the state schema, node functions, and edges.

2

Trace

Follow state from START through nodes to END.

3

Explain

Say why a node runs and how it updates state.

4

Diagnose

Find a wrong route, conflicting update, or missing join.

5

Modify

Add a node or edge and predict the new path.

Goal: Implement workflows with LangGraph

SECTION 1

LangGraph runs a workflow

Code or a model can choose the next step

CONTROL METHOD

Who chooses the next step?

application code	model

This is a control choice; it does not determine which runtime executes the workflow.

Agent topology is separate from step-by-step control

CONTROL METHOD

Who chooses the next step?

- application code
- model

AGENT TOPOLOGY

How is the reasoning work organized?

- no agent
- one agent
- manager + specialists
- handoffs or peers

Who chooses steps and how agents collaborate are independent decisions.

LangGraph executes the graph, not the agent design

CONTROL METHOD

Who chooses the next step?

- application code
- model

AGENT TOPOLOGY

How is the reasoning work organized?

- no agent
- one agent
- manager + specialists
- handoffs or peers

EXECUTION RUNTIME

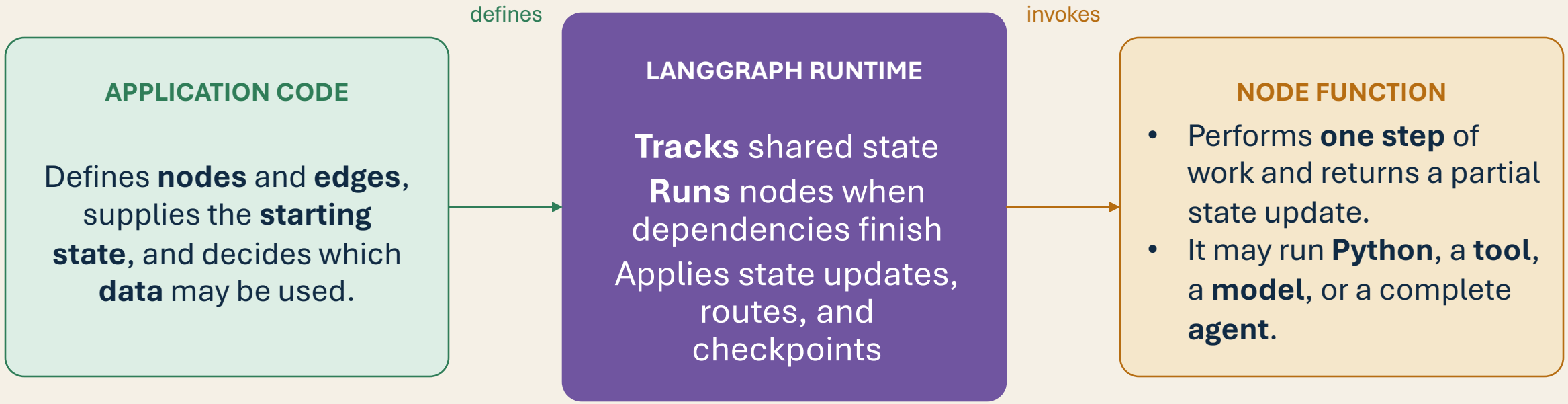
Which runtime runs and saves the graph?

- plain Python
- OpenAI Agents SDK
- **LangGraph**

LangGraph coordinates graph execution, state, and routes; optional checkpoints support resuming a run.

LangGraph runs the graph that your application defines

Three responsibilities, connected by explicit inputs and outputs



A node runs only after its required incoming dependencies finish.

LangGraph workflows can use zero, one, or many agents

Legend: rounded box = LangGraph node. Arrow = edge that allows another node to run.

ZERO AGENTS

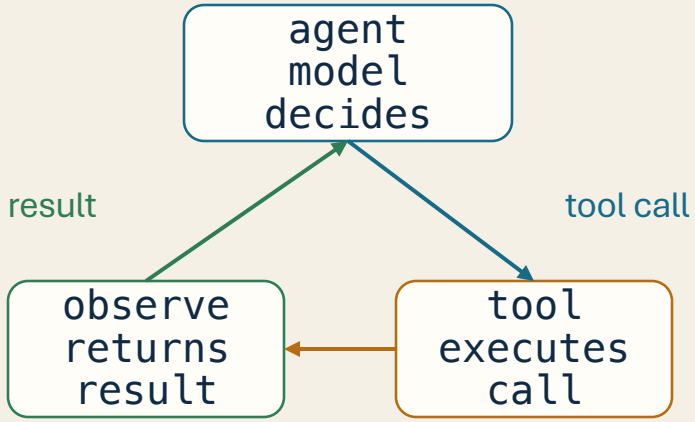
deterministic workflow



All three nodes contain ordinary **Python**.

ONE AGENT

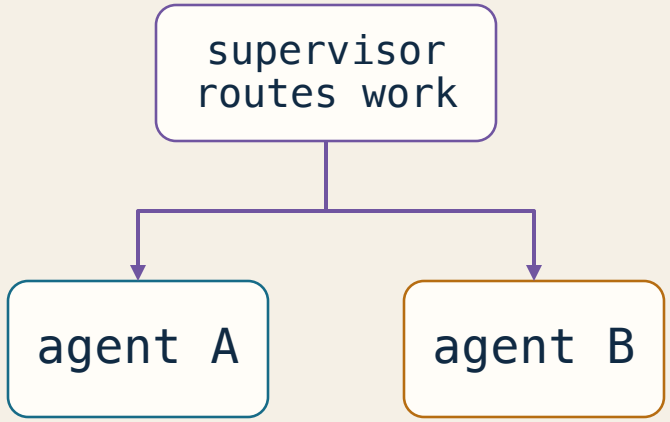
model and tool loop



The agent node chooses when to call the **tool** node.

MANY AGENTS

supervisor, handoff, or network



The supervisor and worker boxes are graph nodes that contain **agents**.

LangGraph vs. OpenAI Agents SDK: when to use each

COMPARISON	LANGGRAPH	OPENAI AGENTS SDK
Basic idea	Connect steps that share information.	Define an agent; the SDK manages its model and tool calls.
How work moves	Use graph edges to route steps and tool calls.	The SDK manages tool calls and agent handoffs.
Saving progress	Save progress; pause and resume a workflow.	Sessions remember context; runs can resume; storage can be added.
Several agents	Put agents in steps and control how they connect.	A lead agent can delegate or hand off tasks.
When to use it	Use it to control steps, share data, and resume work.	Use it for a built-in agent loop, tool calls, and handoffs.

Both can call tools and coordinate agents; you can also use them together.

SECTION 2

LangGraph runs nodes over shared state

Track state updates, routing decisions, and the END transition.

The smallest runnable graph increments a counter once

This graph reads count, increments it once, and then stops.

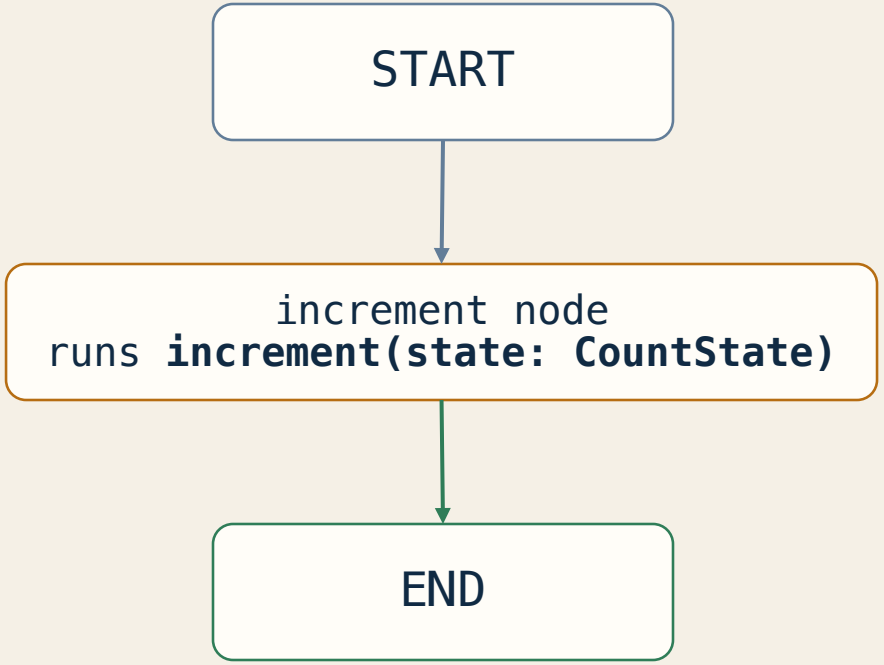
RUNNABLE PYTHON

```python
from typing import TypedDict
from langgraph.graph import START, END, StateGraph

class CountState(TypedDict):
    count: int

def increment(state: CountState):
    return {"count": state["count"] + 1}

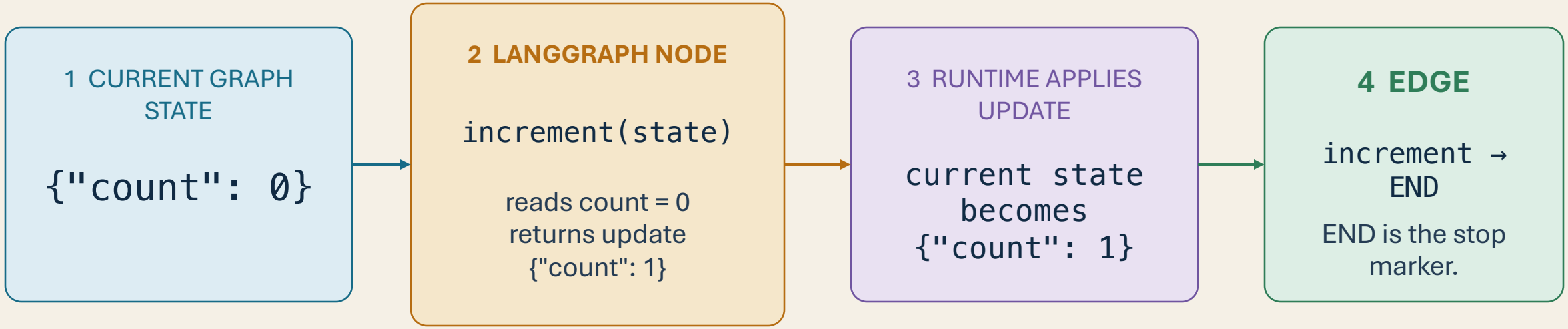
builder = StateGraph(CountState)
builder.add_node("increment", increment)
builder.add_edge(START, "increment")
builder.add_edge("increment", END)
graph = builder.compile()
```



compile() prepares the graph object that invoke() will run.

One graph step changes state, then an edge selects what can run next

Graph state means the current shared values for one invocation of the graph.



A node reads current graph state and returns an update. LangGraph applies it, then follows an edge.

Describe and compile the graph once, then invoke it with an initial state

The builder fixes nodes and edges; each invocation supplies fresh initial state.

DESCRIBE ONCE

SETUP

```
builder = StateGraph(CountState)
builder.add_node("increment", increment)
builder.add_edge(START, "increment")
builder.add_edge("increment", END)

graph = builder.compile()
```

Declare nodes and edges. compile() checks the definition and creates the executable graph.

RUN EACH REQUEST

EXECUTION

```
initial_state = {"count": 0}
result = graph.invoke(
    initial_state
)
```

invoke(initial_state) returns the final state.

The builder describes the graph once. Each invoke() call starts a new run from its initial state.

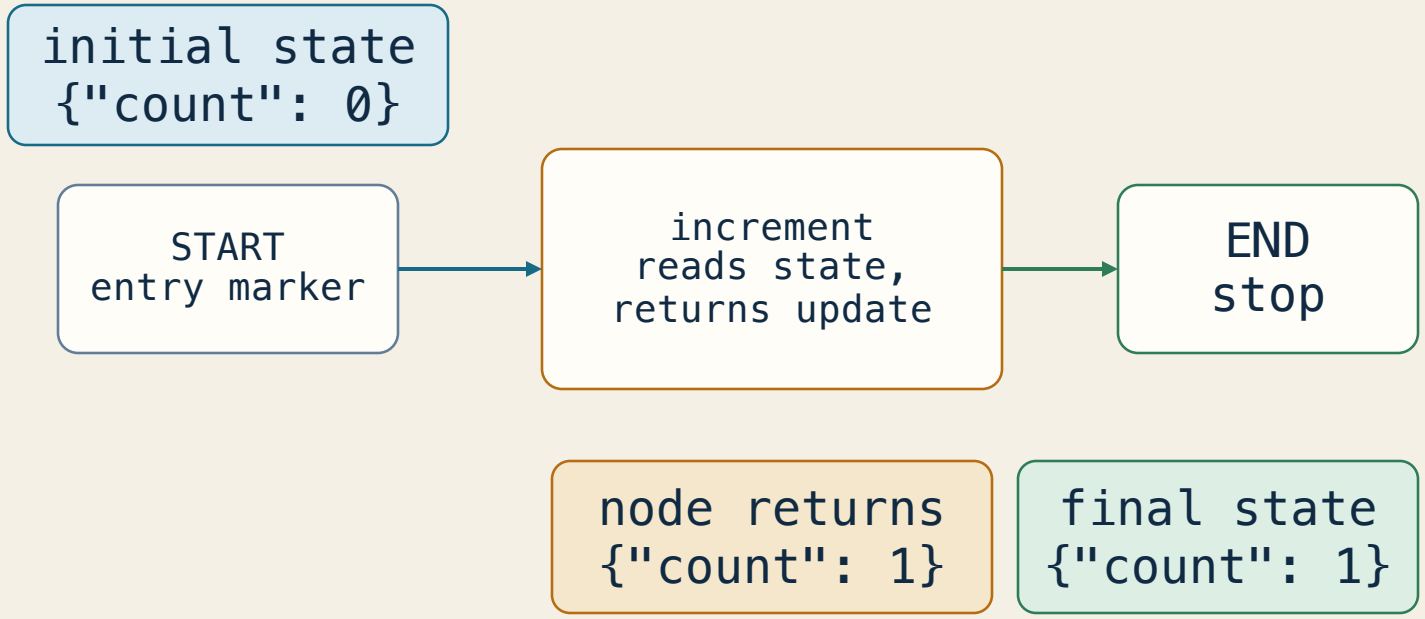
One invocation changes count from 0 to 1

The graph topology stays START → increment → END while the state value changes along that path.

ONE INVOCATION

```
initial = {"count": 0}
result =
graph.invoke(initial)

# result
{"count": 1}
```



LangGraph applies the returned update before following the edge from increment to END.

The boxes are the actual graph. The labels show the state observed or produced at each step.

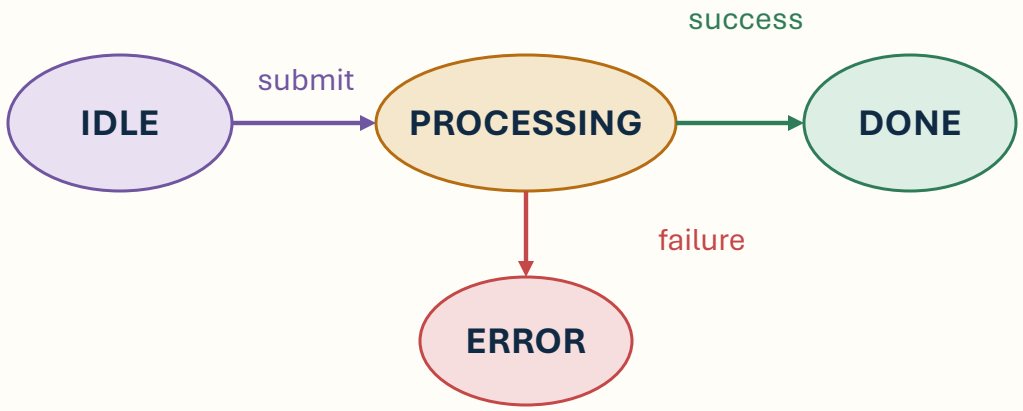
FSM vs. LangGraph: the nodes mean different things

An FSM node is a system state. A LangGraph node is an action that reads and updates a separate data state.

CLASSIC FINITE-STATE MACHINE

NODE = STATE

CONTROL STATES



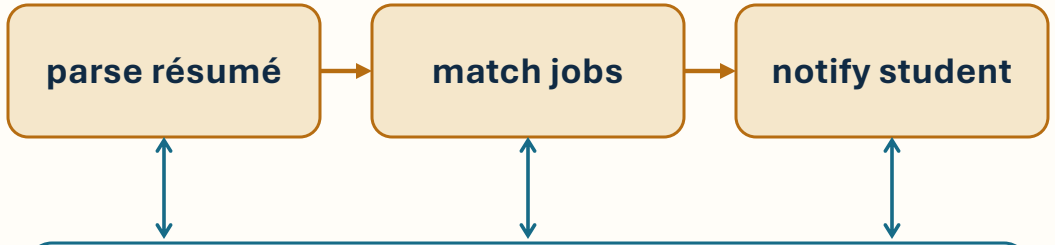
The active node is the system's current condition.

An event moves control to another state.

LANGGRAPH

NODE = ACTION

CONTROL FLOW , which action runs next



GRAPH STATE , separate data payload

`{resume, profile, matches, status}`

Each node reads this object and returns a partial update.

Edges route execution; they do not contain the data state.

FSM asks "Which mode are we in?" LangGraph asks "Which action runs next, and how does it update the shared data?"

A LangGraph can implement FSM-like routing, but its nodes are computations, not the graph-state payload.

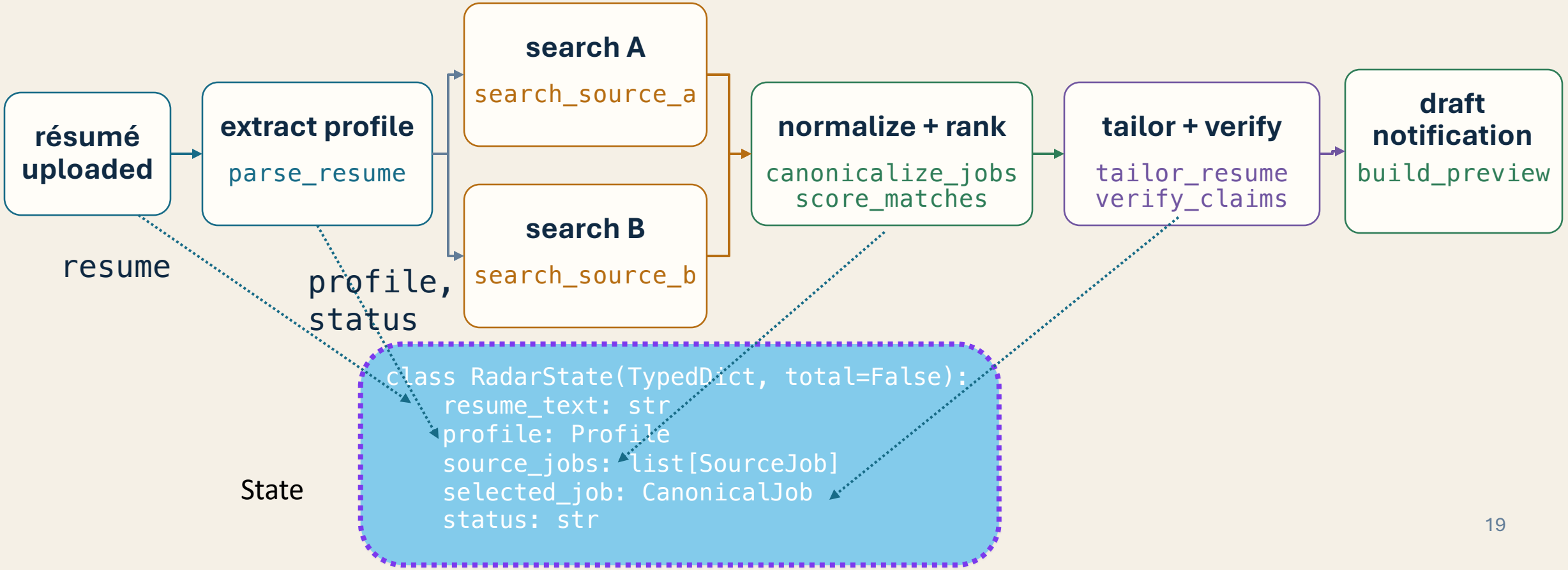
SECTION 3

Example: Job Radar for résumé matching

Follow a student résumé through search, ranking, claim checks, and review.

Running example: Job Radar matches students to jobs

After a student uploads a résumé, parse_resume extracts a profile from its text; no saved profile is assumed. Job Radar searches two job sources in parallel, ranks matches, checks résumé claims, and drafts a notification for review.



A state schema names the shared fields for one graph run

The graph starts with text parsed from the uploaded résumé. parse_resume extracts the profile; later nodes add jobs and status.

ABRIDGED STATE SHAPE

```
class RadarState(TypedDict, total=False):
    resume_text: str
    profile: Profile
    source_jobs: list[SourceJob]
    selected_job: CanonicalJob
    status: str
```

- **TypedDict** is a type hint for the allowed keys and value types in an ordinary Python dict.
- **total=False** means a key may be absent until a node adds it.

Profile

Structured facts extracted from the résumé text.

list[SourceJob]

Raw job records returned by the search sources.

CanonicalJob

One normalized, deduplicated job record produced before ranking selects it.

str

A short progress value such as searching or matching.

Pydantic BaseModel validates at runtime; TypedDict provides lightweight static typing.

A node returns key-value updates, and LangGraph applies them to state

parse_resume reads résumé text from state, extracts a new profile, and returns it as an update. No saved profile is assumed.

NODE UPDATE EXAMPLE

```
def parse_resume(state: RadarState):
    text = state["resume_text"]
    profile = extract_profile(text)
    return {
        "profile": profile,
        "status": "searching",
    }
```

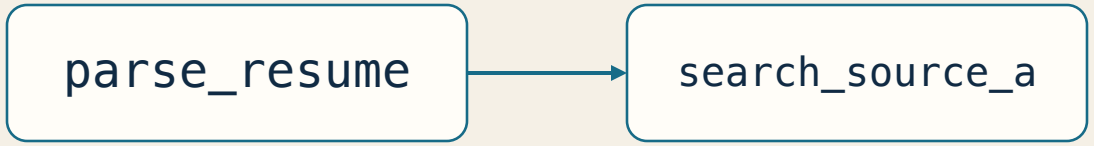
The returned dict is a partial update: it names two state keys and their new values.

KEY	BEFORE	UPDATE	AFTER
profile	absent	extracted	equivalent profile
status	absent	"searching"	"searching"

LangGraph applies each returned key-value pair to the matching state key.

Three edge patterns: always continue, choose a route, or stop at END

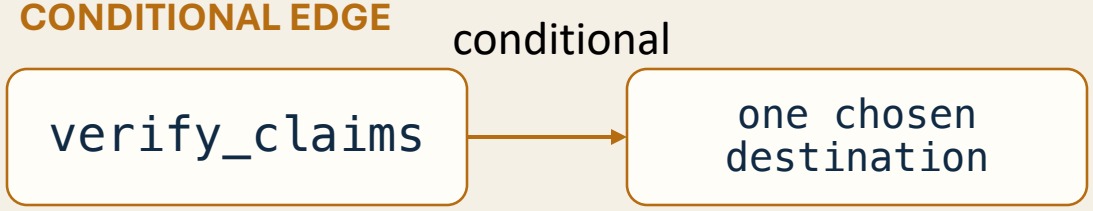
FIXED EDGE



After parse_resume, fixed edges start search_source_a and search_source_b together.

```
add_edge("parse_resume",
         "search_source_a")
```

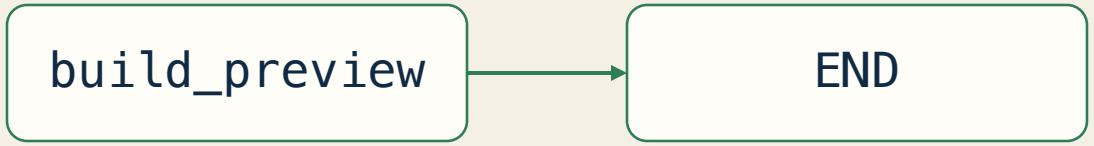
CONDITIONAL EDGE



After the node update is applied, a routing function chooses build_preview or END.

```
add_conditional_edges(
    "verify_claims", route_after_verification)
```

TERMINAL EDGE



This is a fixed edge whose destination is the special END marker. END stops the run.

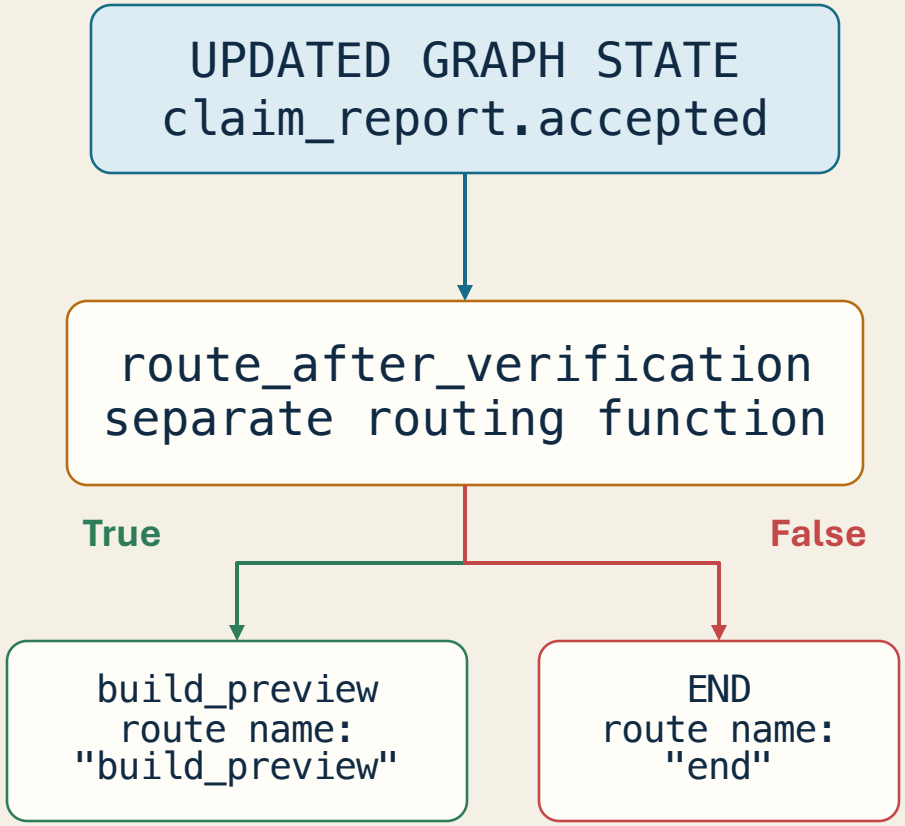
```
add_edge("build_preview", END)
```

Conditional-edge routing keeps the node update and route decision separate

RUNNABLE CONDITIONAL EDGE

```
def route_after_verification(state):
    report = state["claim_report"]
    accepted = report["accepted"]
    if accepted:
        return "build_preview"
    return "end"

builder.add_conditional_edges(
    "verify_claims",
    route_after_verification,
    {"build_preview": "build_preview",
     "end": END},
)
```



The node has already returned its update. This separate function now reads the updated state and returns a route name.

Conditional edges read state; Command routes inside the node

CONDITIONAL EDGE

Use a **conditional edge** when verify_claims writes claim_report; a separate router then reads that state to choose the next node.

SEPARATE ROUTER

```
def route(state):
    report = state["claim_report"]
    if report["accepted"]:
        return "build_preview"
    return END

builder.add_conditional_edges(
    "verify_claims", route)
```

Use it when a separate function can route from fields already saved in graph state.

COMMAND

Use **Command** when **verify_claims** decides where to go as it checks claims; return the update and destination together.

ABRIDGED COMMAND

```
def verify_claims(state):
    ok = check_claims(state)
    goto = END
    if ok: goto = "build_preview"
    return Command(
        update={"claim_report":
                {"accepted": ok}},
        goto=goto)
```

Use it when the node itself knows the next destination as it computes the update.

Pitfall: a static outgoing edge also runs if the node returns Command(goto=...).

Streaming updates show each node’s result as it finishes

Streaming updates lets the caller receive each node’s returned partial update as soon as that node finishes. The caller does not wait for the whole graph to reach END.

```
for event in graph.stream(
    initial_state, stream_mode="updates"):
    print(event)
```

Illustrative updates, one possible search order

1

parse_resume

```
{parse_resume: profile=extracted, status=searching}
```

2

search_source_a

```
{search_source_a: source_jobs[2]}
```

3

search_source_b

```
{search_source_b: source_jobs[2]}
```

Each update names its node and fields. Parallel events may arrive in either order.

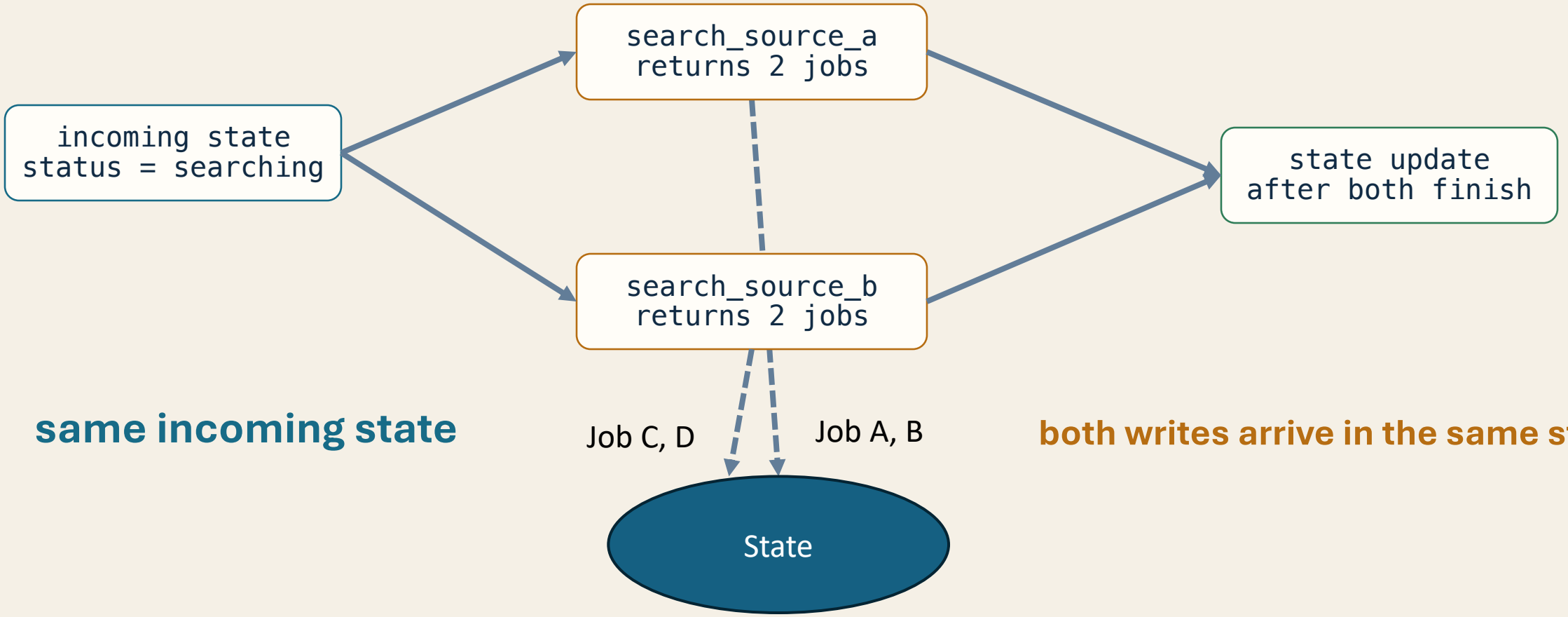
SECTION 4

Parallel nodes may write the same state field

No race condition!
Reducers combine concurrent updates; joins release downstream nodes.

Parallel nodes read the same incoming state

A superstep is one graph step in which every ready node reads the same incoming state.



Predict the result of two writes to source_jobs

```python
class BrokenState(TypedDict):
    source_jobs: list[dict]
```

source_a returns
{"source_jobs": [A1, A2]}

source_b returns
{"source_jobs": [B1, B2]}

What value reaches source_jobs?

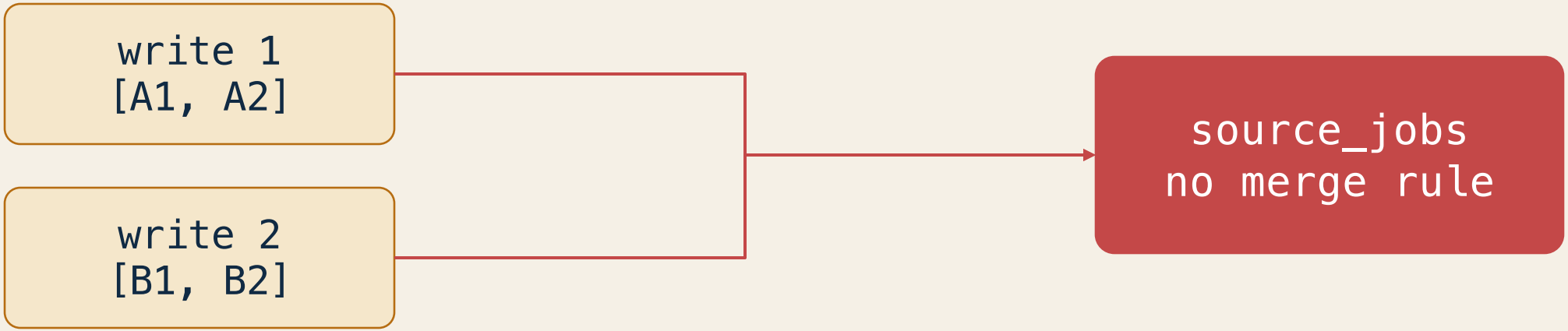
- **A** **A1, A2 only**
- **B** **B1, B2 only**
- C [A1, A2, B1, B2]
- **D** **InvalidUpdateError**

Choose an answer and name the merge rule you assumed.

Two writes without a reducer raise InvalidUpdateError

InvalidUpdateError

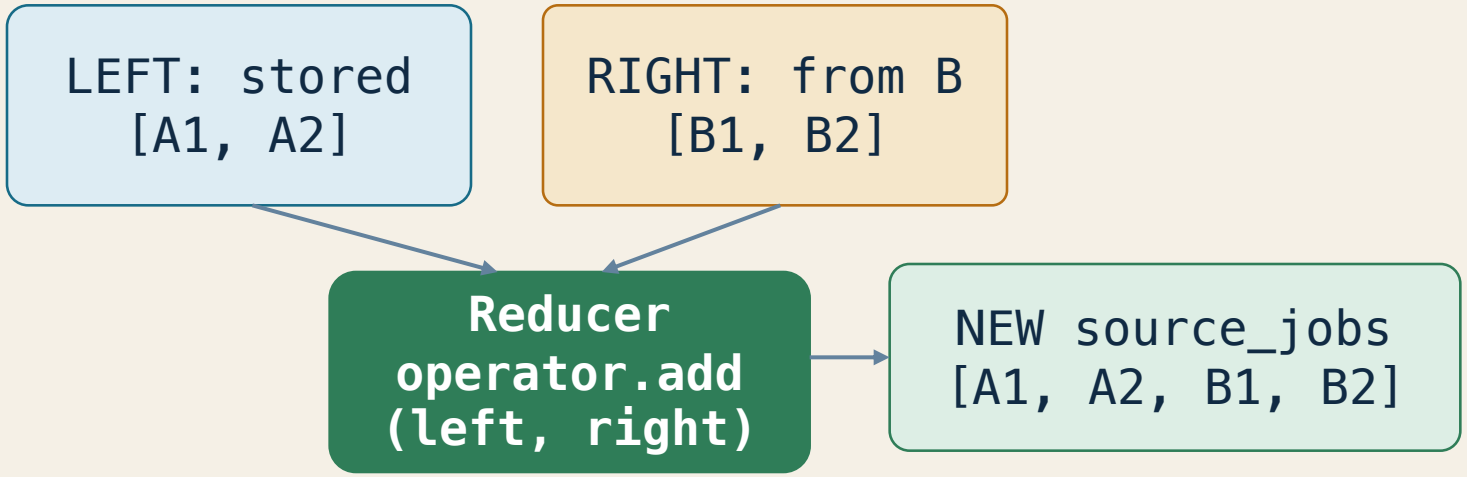
```
At key 'source_jobs': Can receive only one value per step
```



LangGraph needs an explicit rule whenever one graph step produces multiple values for one field.

A reducer defines how concurrent writes combine

operator.add(left, right) joins two lists: the stored source_jobs value and ONE new node update. A then B is only an example; parallel merge order can vary.



STATE KEY + REDUCER

```
source_jobs: Annotated[
    list[SourceJob], operator.add]
```



Parallel Job Radar searches need a reducer to combine results

Job Radar runs search_source_a and search_source_b in parallel. Both write source_jobs; without a reducer, LangGraph rejects the concurrent updates.

JOB RADAR CODE

```
from typing import Annotated, TypedDict
import operator

class RadarState(TypedDict):
    source_jobs: Annotated[
        list[str], operator.add]

def search_source_a(state):
    return {"source_jobs": ["A"]}
def search_source_b(state):
    return {"source_jobs": ["B"]}
```

What the reducer does

`Annotated[list[str], operator.add]` makes source_jobs concatenate the lists returned by both search nodes.

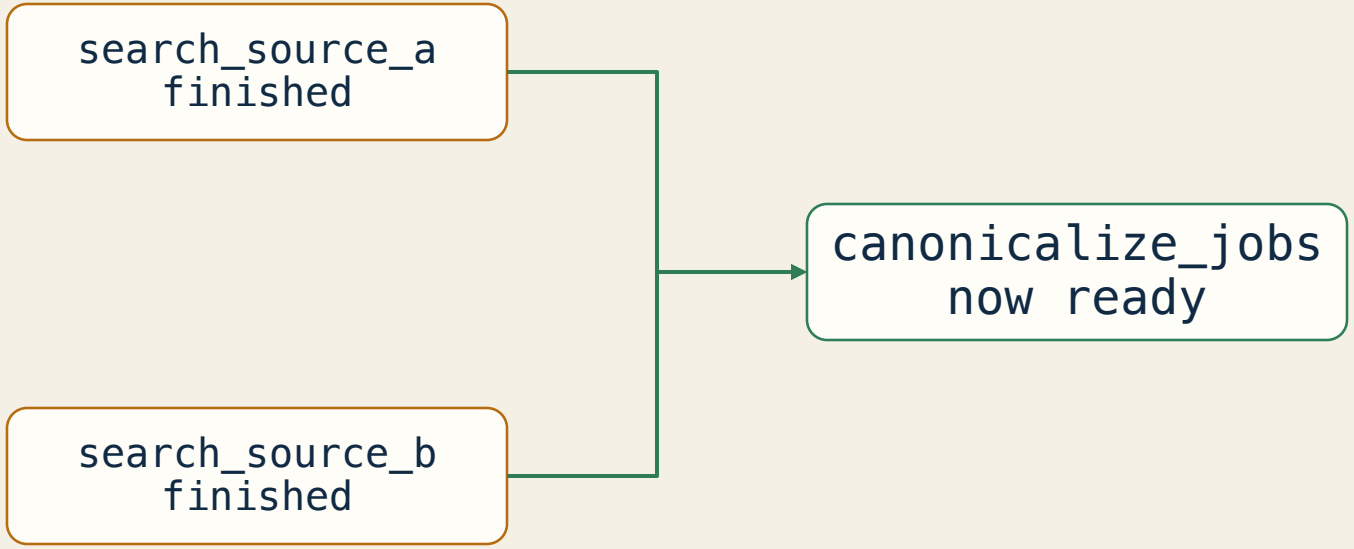
NO REDUCER: source_jobs

```
Two writes in one step
→ concurrent-update error
```

WITH REDUCER: source_jobs

```
A: ["A"] + B: ["B"]
   → ["A", "B"]
```

A join waits for every required predecessor



JOIN EDGE

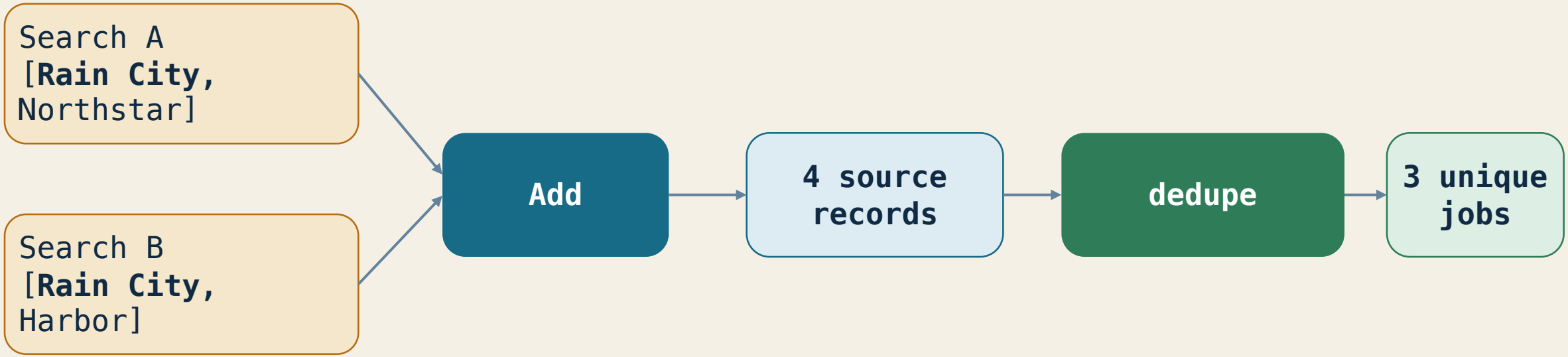
```
builder.add_edge(
    ["search_source_a",
     "search_source_b"],
    "canonicalize_jobs",
)
```

JOIN decides when canonicalize_jobs can run

REDUCER decides how source_jobs combines

Reducer combines search results; deduplication removes repeats

Job Radar collects both result lists, then removes repeated job postings.



operator.add concatenates both search lists, keeping both Rain City records. canonicalize_jobs then removes the repeated posting, leaving three unique jobs.

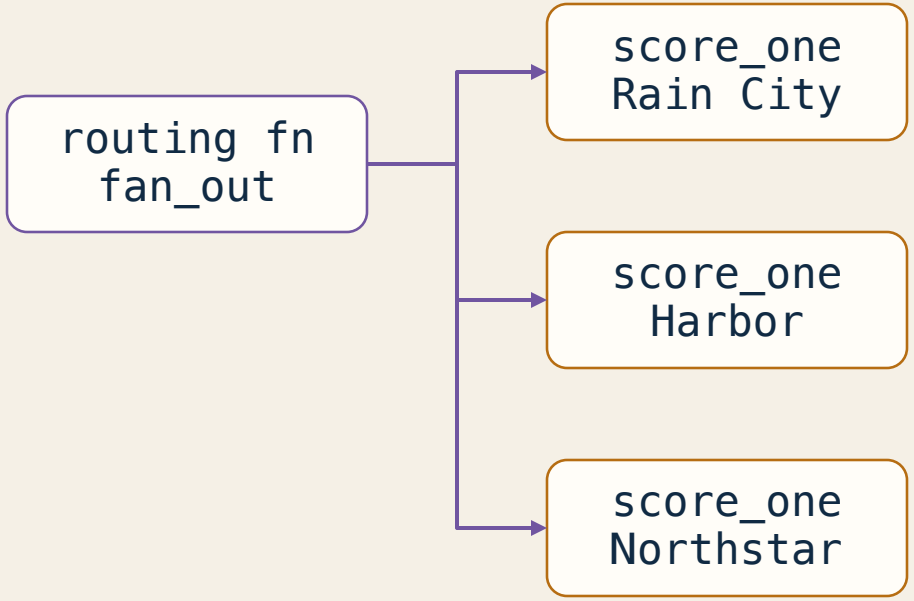
Use Send to score each Job Radar match in parallel

Earlier, **score_matches** ranked the whole list. To score any number of jobs in parallel, fan_out sends each job to a separate score_one task.

DYNAMIC MAP STEP

```
def fan_out(state):
    jobs = state["canonical_jobs"]
    return [
        Send("score_one", {"job": job})
        for job in jobs
    ]

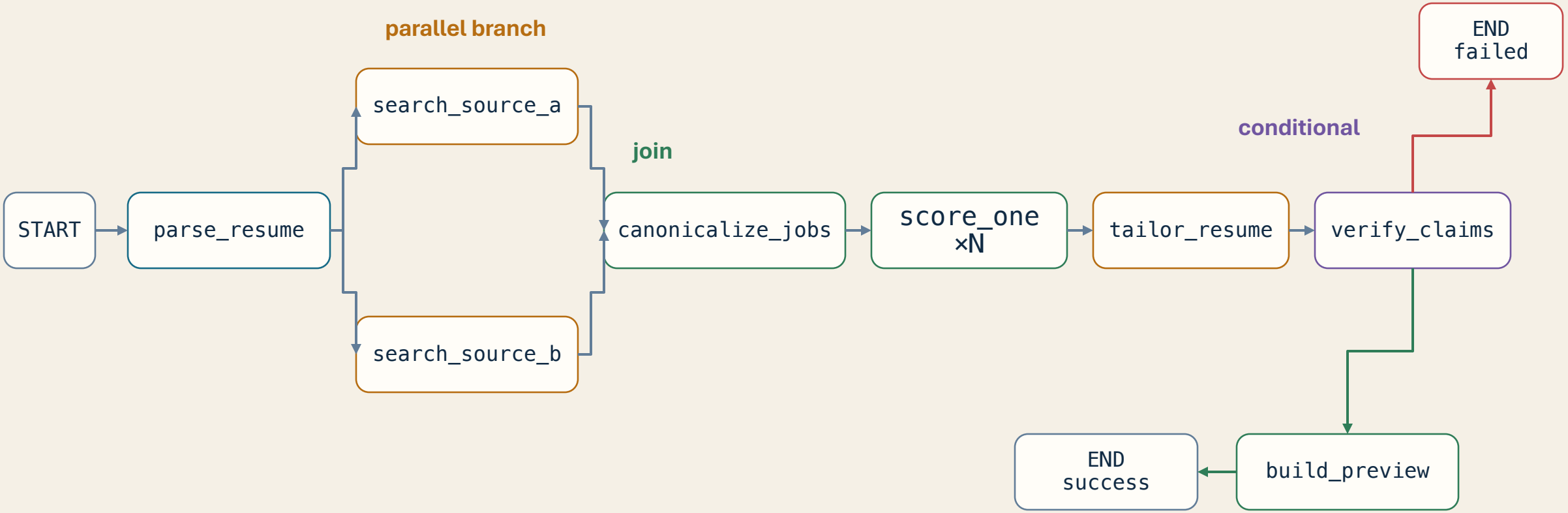
builder.add_conditional_edges(
    "canonicalize_jobs", fan_out
)
```



Each task receives its own job. A reducer combines their score updates.

Job Radar forks search, scores jobs, then checks claims

Each Send starts one score_one task; merge scores, then choose a job to tailor.



score_one ×N compresses scoring, merging, and selecting one job. Both END boxes mean the same stop point.

A node may run code, call a tool, or invoke a model

TO THE RUNTIME, EACH NODE IS STILL A FUNCTION

DETERMINISTIC

`canonicalize_jobs`

Same state → same update

great for tests

TOOL-BOUND

`search_source_a`

Queries a jobs API or database

needs failure handling

MODEL-BACKED

`tailor_resume`

Produces structured proposal

needs validation

Put uncertainty inside a node. Keep its input and output contracts explicit.

One tool call inside search_source_a

The node derives a request from graph state, calls search_jobs, validates the result, then returns source_jobs.

ABRIDGED PYTHON: TOOL CONFIGURED ELSEWHERE

```python
def search_source_a(state: RadarState) -> dict:
    profile = ProfileDraft.model_validate(state["profile"])
    query = " ".join(profile.skills)
    raw_jobs = search_jobs.invoke({
        "query": query,
        "limit": 10,
    })
    jobs = [SourceJob.model_validate(item)
            for item in raw_jobs]
    return {"source_jobs":
            [job.model_dump() for job in jobs]}
```

TOOL REQUEST

```
{"query": "Python ML",
 "limit": 10}
```

UNTRUSTED TOOL RESULT

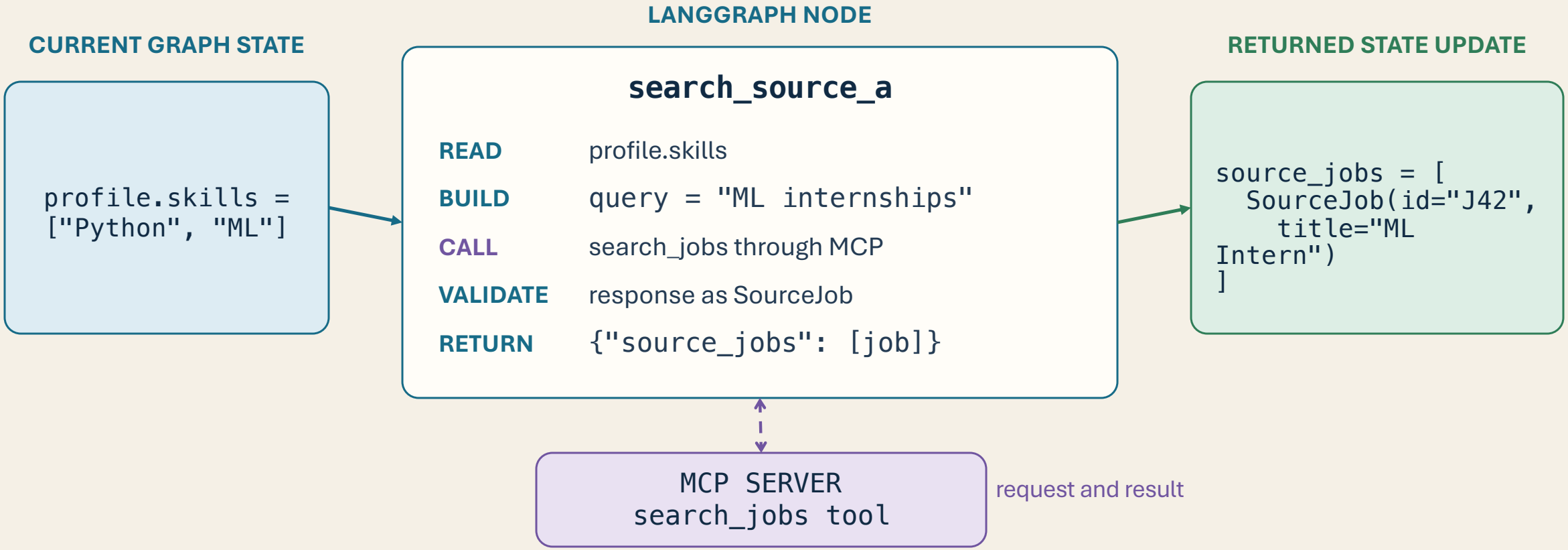
```
{"source_job_id": "J42",
 "company": "Rain City Robotics",
 "title": "ML Intern",
 "location": "Seattle, WA"}
```

VALIDATED NODE UPDATE

```
{"source_jobs": [
  SourceJob(source_job_id="J42",
            title="ML Intern")
]}
```

MCP connects a LangGraph node to an external tool

LangGraph schedules the node and applies its state update. Model Context Protocol (MCP) standardizes the tool request and response.



MCP handles the external tool exchange. LangGraph decides when the node runs and applies the returned update to graph state.

One LLM call inside load_profile

The node reads résumé text, invokes a structured-output model, then returns profile and status for LangGraph to apply.

ABRIDGED PYTHON: MODEL CONFIGURED ELSEWHERE

```python
extractor = model.with_structured_output(
    ProfileDraft
)

def load_profile(state: RadarState) -> dict:
    messages = [
        ("system", "Extract only stated facts."),
        ("user", state["resume_text"]),
    ]
    profile = extractor.invoke(messages)
    return {
        "profile": profile.model_dump(),
        "status": "searching",
    }
```

MODEL INPUT

system: "Extract only stated facts."
user: résumé text from state

PARSED MODEL OUTPUT

```
ProfileDraft(
  skills=["Python", "ML"],
  degree="M.S. EE", grad_year=2027
)
```

NODE RETURNS TO LANGGRAPH

```
{"profile": profile.model_dump(),
 "status": "searching"}
```

The model call lives inside the Python node. LangGraph schedules that node and applies the dictionary it returns.

Sources: LangGraph Graph API; LangChain structured-output documentation

Model-generated résumé claims become validated state updates

A model proposes claims; validation checks their structure before saving.

SCHEMATIC NODE

```python
class ResumeClaim(TypedDict):
    text: str
    claimed_skills: list[str]
    evidence_ids: list[str]

def tailor_resume(state: RadarState) -> dict:
    claim = validate(model(state), ResumeClaim)
    return {
        "resume_claims": [claim],
        "status": "verifying",
    }
```

The graph can now

- route on validation results
- checkpoint the proposal
- test the verifier without a live model
- reject unsupported claims

The model proposes. The graph decides what happens next.

SECTION 5

Checkpoints make recovery traceable

Inspect routes, reload by thread_id, and retry without duplicate side effects.

A checkpointer saves each step under one thread_id

A checkpointer saves graph state after each step. The **thread_id** identifies which run to load.

CHECKPOINT SETUP

```
saver = InMemorySaver()
graph = builder.compile(checkpointer=saver)

config = {"configurable": {
    "thread_id": "radar-run-005"
}}
graph.invoke(initial_state(), config)

current = graph.get_state(config)
```

LOADED SNAPSHOT

```
thread_id = radar-run-005
status = preview_ready
next = []
saved fields = 11
```

next = [] means the run finished. History contains earlier snapshots and their next nodes.

Checkpoints track one thread. Store spans threads

CHECKPOINTER

Graph state snapshots
One thread identified by thread_id
Continuity, interrupts, fault recovery, time travel

STORE

Application-defined key-value data
Available across threads
User preferences, durable facts, shared knowledge

A GRAPH MAY USE BOTH

```
graph = builder.compile(
    checkpointer=checkpointer, store=store
)
config = {"configurable": {
    "thread_id": "radar-run-005"
}}
```

An interrupt pauses a saved run until input arrives

1 PAUSE INSIDE A NODE

```
def approval_node(state):
    approved = interrupt({
        "question": "Send preview?",
        "preview":
state["notification_preview"],})
    return {"approved": approved}
```

2 RESUME THE SAME THREAD

```
config = {"configurable": {
    "thread_id": "radar-run-005"
}}

graph.invoke(initial_state, config)

graph.invoke(
    Command(resume=True), config
)
```

PAUSE

interrupt() returns a payload to the caller

SAVE

the checkpointer records state and the pending task

RESUME

Command(resume=...) becomes the interrupt return value

The resumed node starts again from its beginning. Code before interrupt() may run again.

Retries and resumes can repeat side effects

RETRY POLICY

TRANSIENT TOOL FAILURE

```
policy = RetryPolicy(max_attempts=3)
builder.add_node(
    "search_source_a",
    search_source_a,
    retry_policy=policy,
)
```

IDEMPOTENCY

EXTERNAL SIDE EFFECT

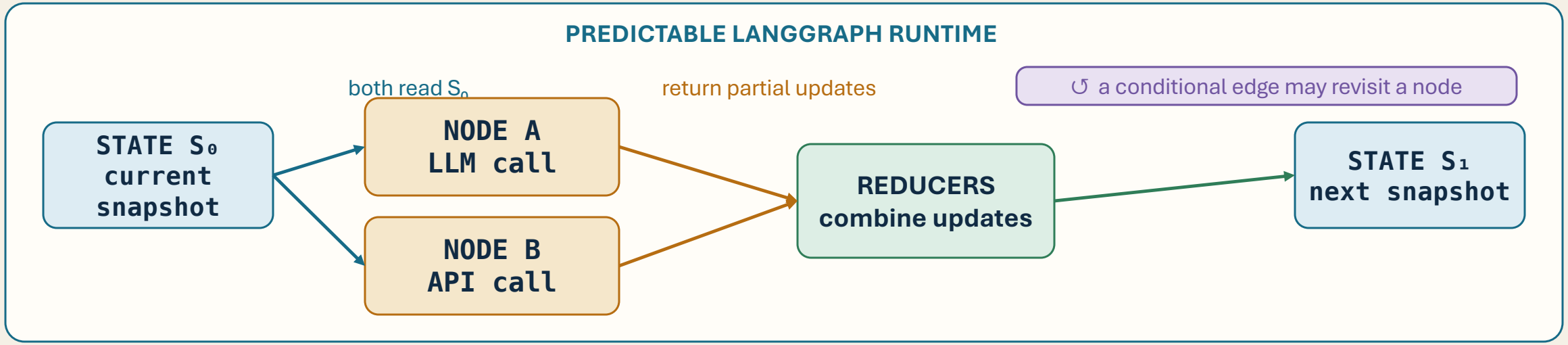
```
key = (
    profile_version,
canonical_job_id,
    "notification"
)
send_notification_once(
    key, preview
)
```

The runtime may rerun a node after a transient error or resume.
The application prevents duplicate external effects.

Summary: Magic of LangGraph is to makes unpredictable agents controllable

In one step, every selected node reads the same state snapshot. LangGraph combines their returned updates before the next step.

ONE SUPERSTEP



CYCLES A conditional edge can revisit an earlier node.

PERSISTENCE A checkpointer can save S_1 when the step finishes.

STREAMING The caller can receive each node update as it completes.

CONTROL FLOW Updated state selects the node(s) for the next step.

LangGraph wraps a predictable state machine around an unpredictable LLM, combining application reliability with agentic reasoning.

Source: LangGraph documentation , Graph API, persistence, and streaming

Test node updates, routes, and saved state separately

Invariant

An unsupported résumé claim must end the run without building a preview.

ROUTE TEST

```
state = {
    "claim_report": {"accepted": False}
}

route = route_after_verification(state)
assert route == "end"
```

Bug this catches

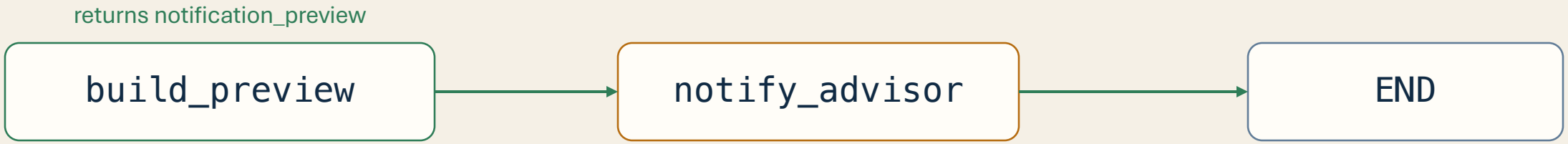
A refactor accidentally routes failed verification to build_preview.

The maintained suite also tests one node update and checkpoint lookup by thread_id.

A useful graph test names the state, the route or update, and the regression it prevents.

Exercise: add one node and predict the new trace

Add a notify_advisor node after build_preview



1. What state field does the node read?
2. What partial update does it return?
3. **Which edge changes?**
4. Do we need a reducer? Why?
5. What would the checkpointer save?

Appendix: complete RadarState

STATE SCHEMA

```
class RadarState(TypedDict, total=False):
    radar_run_id: str
    resume_text: str
    profile: Profile
    source_jobs: Annotated[list[SourceJob], operator.add]
    canonical_jobs: list[CanonicalJob]
    matches: list[MatchResult]
    selected_job: CanonicalJob
    resume_claims: list[ResumeClaim]
    claim_report: ClaimReport
    notification_preview: dict[str, object]
    status: str
    timeline: Annotated[
        list[dict[str, object]], operator.add
    ]
```

Why this stays in the appendix

INPUT

```
radar_run_id
resume_text
```

INTERNAL

```
profile
source_jobs
claim_report
```

OUTPUT

```
notification_preview
status
```

Input and output schemas may expose subsets of this internal state.

Appendix: MessagesState and add_messages

Conversational graphs often need message history plus application-specific state.

PREBUILT MESSAGE CHANNEL

```python
from langgraph.graph import MessagesState

class State(MessagesState):
    documents: list[str]
    approved: bool

# MessagesState includes:
# messages: Annotated[
#   list[AnyMessage], add_messages]
```

add_messages

New message ID	append the message
Existing message ID	replace that message
Dict input	deserialize to message objects

MessagesState is a useful default for chat history. It does not replace domain state.

Stream modes expose state, updates, messages, and tasks

MODE	QUESTION IT ANSWERS
`values`	What does the full state look like after each step?
`updates`	Which partial update did each node produce?
`messages`	Which model tokens and metadata are arriving?
`custom`	What application progress did a node emit?
`checkpoints`	Which graph snapshots were saved?
`tasks`	Which tasks started, finished, or failed?
`debug`	What complete runtime detail is available?

Version 2 streaming returns a uniform part with type, namespace, and data fields.

Sync and async nodes share the same graph state and edges

SYNCHRONOUS

```
def search_source_a(state):
    jobs = client.search()
    return {"source_jobs": jobs}

result =
graph.invoke(initial_state)
```

ASYNCHRONOUS

```
async def search_source_a(state):
    jobs = await client.search()
    return {"source_jobs": jobs}

result = await
graph.ainvoke(initial_state)
```

Same state field

Same graph edges

Different Python execution style

Changing a node to async does not change the state, reducer, or topology.

Extend LangGraph after mastering its execution basics

The lecture established the graph execution model. These features extend it rather than replace it.

Subgraphs	Package a graph as a node and compose larger systems.	**Functional API**	Use durable execution with ordinary function structure.
Caching	Reuse node results when inputs and policy allow it.	**Recursion limits**	Stop a graph after a chosen number of loop steps.
Production persistence	Replace in-memory savers and stores with durable backends.	**Observability**	Trace and evaluate runs with LangSmith or another tracing system.

Add these after you can trace state, routes, supersteps, and checkpoints.